

ARI: 自律型研究基盤

樹神 宇徳

スナップショット v0.7.1, 2026

Abstract

本レポートでは、木探索ベースの探索ループと論文生成パイプラインを組み合わせた自律型研究基盤 **ARI** を述べる。探索ループは研究ステップノードに対する最良優先木探索 (BFTS) であり、ノード拡張は ReAct 形式のエージェントが 14 のドメインスキルレジストリにツール呼び出しを発する形で行う。論文生成パイプラインは生き残った系譜から草稿を生成し、階層型ルブリックの下で反復的に改稿する。すべての成果物は単一のチェックポイントディレクトリに格納され、これが再現性、系譜、コスト計上の単位となる。本レポートはアルゴリズムを形式化し、実装をソースコードレベルで記述し、評価スナップショットを提示し、設計を制約する決定性と新規性のトレードオフを議論する。

1 はじめに

1.1 動機

機械学習および周辺領域の研究ワークフローには共通の骨格がある: 研究アイデアを発想し、実験を設計・実行し、結果を分析し、論文を執筆し、そして反復する。各ステップは大規模言語モデルにとってますます扱いやすくなっているが、それらを単一の自律ループへ縫合することは、これまで未解決のエンジニアリング課題であった。最も近接する既発表システム — AIDE、AI Scientist ファミリ、VirSci、PaperBench 評価器 — は、それぞれ 1~2 軸を埋めはするが、残りの軸は人手に任せている。

ARI はこの空隙を埋めるために存在する。本稿ではユーザが研究問題と計算予算を与えると仮定する; システムは、発表可能な論文、各主張を裏付ける実験コード、そして到達経路の完全な監査記録を返す。監査記録は譲歩できない要件である: 探索木の各ノード、各ツール呼び出し、各モデル呼び出し、そして API 利用の各ドルが、単一のチェックポイントディレクトリ内に記録される ([2] は同様の規律を採用している)。チェックポイントは再現性の単位である。

1.2 貢献

本レポートは以下の三つの貢献を形式化する:

- 研究ステップノード上の**最良優先木探索**であり、その選択スコア $U(n)$ は経験報酬、新規性項、深さペナルティを分離している (section 3.1)。実装は `ari-core/ari/orchestrator/bfts.py:114` にある。
- 階層型ルブリック R をキーに据える**論文生成パイプライン**; 葉判官が個々の主張を採点し、システムスコアは $\text{score}(P) = \sum_i w_i \text{judge}_i(P)$ となる (section 4.2)。PaperBench 複製器はツール呼び出しとして統合される (section 4.4)。
- パイプラインを完全に再現可能にする**チェックポイントスコープ**の規律: すべての外部状

態 (メモリ、コスト台帳、系譜ポインタ) は `$ARI_CHECKPOINT_DIR` 配下にかかれ、ユーザのホームディレクトリには決して書き出されない。これは P2 決定性原則の運用上の帰結である (section 2.5)。

1.3 本レポートの適用範囲

スナップショットは `ari-core` および 14 個の `ari-skill-*` パッケージの v0.7.1 で凍結する。実装引用はファイル名と行番号を併記する; 読者は section 2.1 の配置図を介してリポジトリと突合できる。本来この版で section 5 に並ぶはずの数値結果は、参照すべき凍結チェックポイントがまだ作成されていないため未提供であり、当該章は「保留」と明記してある。表紙にスナップショットバージョンを記すのは、本レポートが行うどの主張も後続のコード修正によって遡及的に無効化されないようにするためである。

1.4 構成

Section 2 は ARI のトップダウンビューを与える: オークストレータ、14 個のスキル、パイプライン DAG、設計原則。Sections 3 and 4 は二つの技術的核であり、それぞれ形式化された定義と経験的観察を含む。Section 5 は評価スナップショット; section 6 は本研究の位置づけ; section 7 は未解決問題を集約する。

2 システム概観

2.1 フルスタック

Figure 1 は v0.7.1 の ARI フルスタックを描く。システムは一つのドライバと多数のツール提供者に分かれる。ドライバ (`ari-core`) はオークストレータ、BFTS エンジン、系譜歩行者、エージェントループ、チェックポイント、コストトラッカ、設定ローダを保有する。各スキルは独立した Python パッケージとして MCP サーバ形式で出荷され、固有の `skill.yaml` 宣言を持つ。現在ツリーに含まれる 14 の

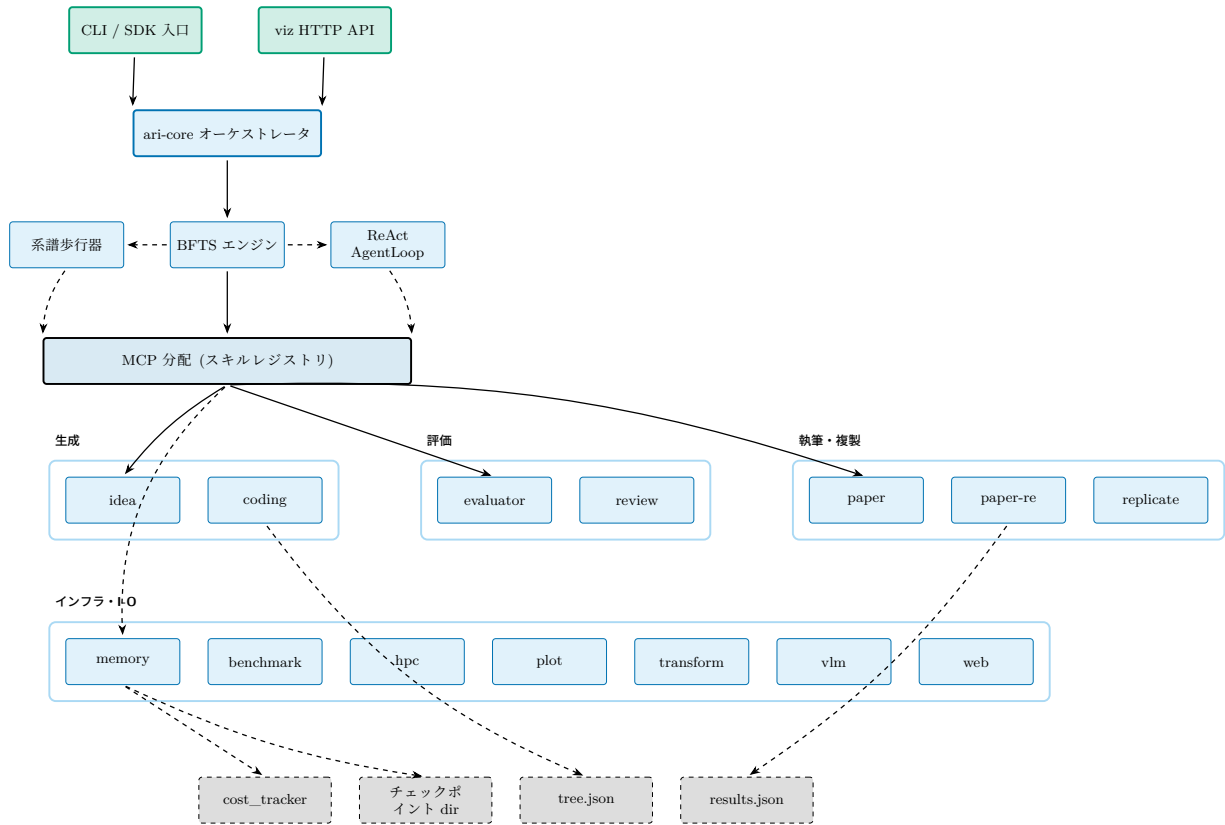


Figure 1: v0.7.1 における ARI のフルスタック。CLI / SDK 入口は単一のオーケストレータ (ari-core) を駆動する; オーケストレータは三つのエンジン (BFTS、系譜歩行器、ReAct AgentLoop) を所有し、MCP 層を通じて 14 スキルレジストリへ分配する。永続化はすべて単一のチェックポイントディレクトリに落ちる: コスト台帳、tree.json、results.json。実線は制御、破線は状態を表す。

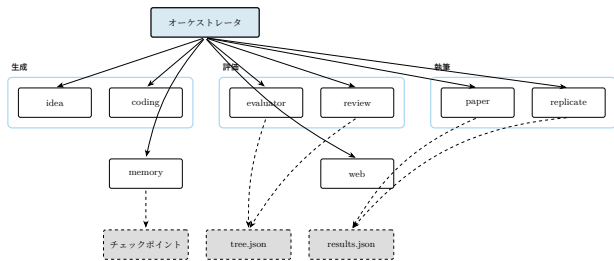


Figure 2: 運用ビュー: オーケストレータがスキルレジストリに分配する; 各スキルはアクティブなチェックポイントディレクトリのみへ書き込む。破線は状態を表す。

スキル: benchmark、coding、evaluator、hpc、idea、memory、orchestrator、paper、paper-re、plot、replicate、transform、vlm、web。

オーケストレータは呼び出しの分配のみを担い、ドメイン作業自体は実行しない。この分離が重要なのは、スキルを追加・差し替えても探索アルゴリズムには触れずに済むからである: ari-core/ari/orchestrator/bfts.py:114 の BFTS コードは、どのスキルが参加するかを知らない; すべては MCP 分配層を介する。

Figure 2 は本レポートの残りで使う「運用ビュー」(制御 + 状態出力)であり、fig. 1 は「層別ビュー」(ドライバ → MCP → スキル → 永続化)である。

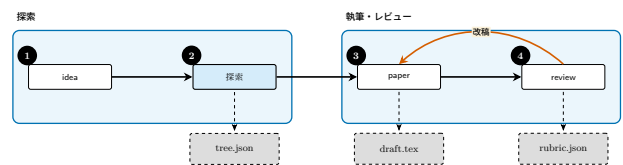


Figure 3: パイプライン DAG: idea → exploration → paper → review、ただし 改稿 がバックエッジを閉じる。破線出力はチェックポイントへ永続化される; 改稿サイクルは草稿を書き直し、レビューを再実行する。

2.2 パイプライン DAG

一回の実行は四つのステーションのパイプラインに帰着する (fig. 3)。idea ステーションは研究問題を選択または生成し、exploration ステーションは BFTS の下でノードを派生させ、paper ステーションは最良系譜を草稿に編成し、review ステーションはルブリックで草稿を採点する。グラフは DAG にレビューから紙へのバックエッジを 1 本加えた構造である: このエッジのみが非単調性を生じうる (改稿が軸を悪化させ得る); 収束基準 (section 4.3) がバックエッジ回数の上界を与える。

パイプライン実行器は ari-core/ari/pipeline/orchestrator.py:131 (run_pipeline); 探索出力と論文入力の間隙を埋める科学データ組立器は同ファイル 68 行目の build_scientific_data である。

2.3 BFTS 制御フロー

Figure 4 は 1 BFTS ステップを端から端まで追う。実装は section 3 で展開する; 図は本レポートの残りです。使う語彙 (フロンティア、選択、フォールバック、expand、prune、停滞) と、各 LLM 主導ステップを実体化するプロンプトファイル (bfts_select.md、bfts_expand_select.md、bfts_expand.md) を確定する。

2.4 paper-re コンポーネントビュー

Figure 5 は paper-re の二段階目のビューである。この層での ARI の貢献は小さいが具体的である: AriPBSolver サブクラスは `_get_tools(_BoundedOutputTool` を挿入するため) と `_run_agent(docker sanity-check` を迂回し `vendor` パスを適合させるため) のみオーバーライドする; それ以外はすべて PaperBench upstream を逐語的に流れ、これにより ARI は upstream の複製タスク枠組みに整合する。第 4 章で図を詳細に追う。

2.5 チェックポイントスコープと設計原則

ARI は 5 つの設計原則 (P1–P5) を中心に構築されている。最も拘束的なのが **P2: 決定性** である。すべての乱択ステップは乱数シードを記録し、すべての LLM 呼び出しはモデル同一性、プロンプト、出力を記録し、すべてのツール呼び出しは `cow_node_id` を伴うことで、書き込みが正しいノードの作業ツリーへ落ちることを保証する。チェックポイントを再走させれば、時計依存またはモデル側非決定性として明示的にマークされた領域を除き、バイト単位で同一の成果物を再現しなければならない。

その他の原則は次の通り:

- **P1 単一成果物ツリー:** 実行のすべての出力は `$ARI_CHECKPOINT_DIR` 配下に存在する。`$HOME` や `/tmp` へ漏出しない。
- **P3 小コンポーネント:** 各スキルは独立パッケージである; スキル交換のために `ari-core` を再構築する必要はない。
- **P4 明示的系譜:** 各ノードと各チェックポイントは親を記録するため、派生実験は差分のみを再計算できる。
- **P5 コスト透明性:** `cost_tracker.py` は各モデル呼び出しにドルコストを付与し、利用者が実行を予算で制限できるようにする。

チェックポイント直列化器は 3 つのトップレベルファイルを書き出す。完全な探索木は `checkpoint.py:49` を経て `tree.json` に入り、ノード単位のペイロードは `checkpoint.py:59` を経て `nodes_tree.json` に入り、実行末の集計は `checkpoint.py:64` を経て `results.json` に入る。チェックポイントを跨ぐ系譜歩行は `lineage.py:126(walk_ancestor_ckpts)` が担い、子から先祖へと辿る。

Figure 6 はチェックポイントのオンディスク形状を描く: 左に共有の実行レベルファイル、右にノード別作業ツリー。この形状が契約である: ARI の出力を入力

にとる任意のツール (後続実行、リグレッションチェック、論文草稿) は、このような単一ディレクトリを指せばよい。

3 探索アルゴリズム

3.1 形式化

探索ループは木 $\mathcal{T} = (\mathcal{N}, E)$ を維持し、各ノード $n \in \mathcal{N}$ は研究の一ステップ — アイディア改訂、コード変更、ベンチマーク実行、分析など — を表す。ノードは以下を保持する:

- 離散状態 $s(n) \in \{\text{open, eval, done, failed}\}$ (`node.py:10`)、
- `NodeLabel` から取られたカテゴリカルなラベル (`node.py:18`)、
- 軸ごとの評価指標を保持する自由形式 dict `node.metrics`、区別されたスカラー `_scientific_score` $\in [0, 1]$ を含む、
- ノードが provisional なテキストではなく実測値を伴うかを示す Boolean `has_real_data`、ならびに
- 選択プロンプトで使われる帳簿フィールド `depth` と `retry_count`。

軸レベル信号からスカラー `scientific score` への集約は、`Evaluator` プロトコル (`ari-core/ari/protocols/evaluator.py:19`) の任意の実装に委譲される; ARI は固定の線形結合を `pin` しない。プロトコルは BFTS と下流のルブリック評価との唯一の継ぎ目であり、軸をスカラーへ縮約する方法について検索エンジンを `agnostic` に保つ。

ハードな探索予算は単一の定数 `config.max_total_nodes` であり、`bfts.py:should_prune` (line 245) によって強制される。ステップ・コスト予算は BFTS エンジン内ではなく呼び出し側の `cost_tracker.py:77` によって別途強制される。

3.2 ノード選択 (LLM 主導)

ARI は意図的にノードのランキングを閉形式のスカラー効用に **縮約しない**。代わりに、`select_next_node` (line 167, 「次のエージェントステップが操作すべきノードはどれか?」) と `select_best_to_expand` (line 258, 「拡張するに最も値する完了ノードはどれか?」) の双方が候補記述リストを LLM に渡し、0 始まりの単一インデックスを解析して受け取る。各ノード n の候補記述は次の形式の一行:

```
[i] id=..., depth=..., retry=...,
    has_real_data=..., metrics={...},
    summary=..., diversity_bonus=+0.05
```

であり、これを消費するプロンプトは `ari/prompts/orchestrator/bfts_select.md` (選択) および

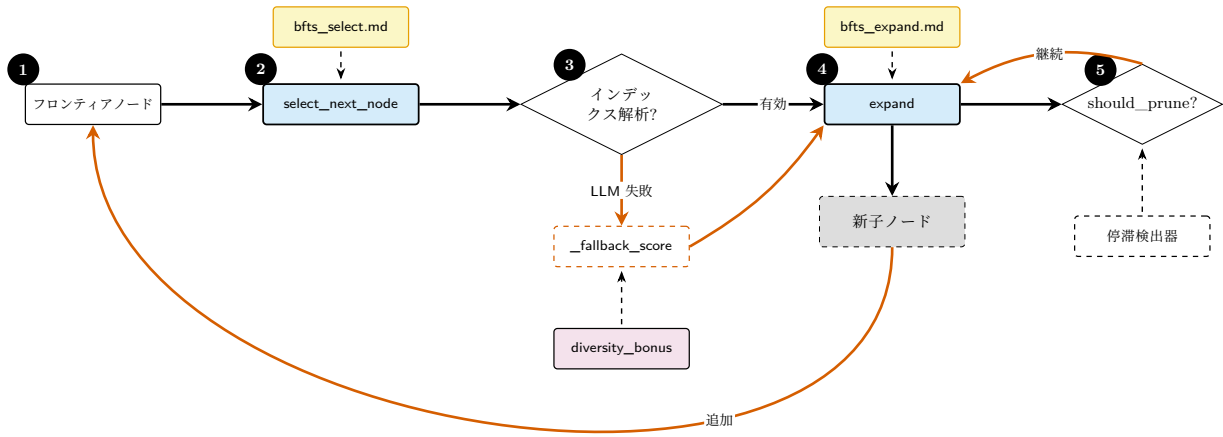


Figure 4: BFTS 制御フロー。LLM 主導の意思決定点は二つ (`select_next_node`、`expand`)、各々別個のプロンプトファイルを参照する; 選択判断は LLM が解析不能なインデックスを返したとき決定的な `_fallback_score` (`_scientific_score` と `diversity_bonus` の和) へフォールバックする。枝刈りは `should_prune`(max-total-nodes ハードカット) と停滞検出器の両方で gating される。詳細は section 3。

ari/prompts/orchestrator/bfts_expand_select.md(拡張3.3 拡張(1 呼び出しにつき 1 子))

張対象) に置かれる。プロンプトが列挙する選択基準は: `has_real_data=True` で強い `metrics` を持つノードは高価値; 未踏方向は試行済みの経路より優先;`metrics` は全体論的に多目的に重み付け; 優れた結果を持つ深いノードは継続に値する; `diversity_bonus` はソフトなタイブレーカとしてのみ扱う。

多様性ボーナス

`BFTS.diversity_bonus`(line 142) は最近のラベルを `_recent_label_history`(`record_run` で埋まるスライディング窓) で追跡する。当該窓内で最頻ラベルに対し相対的に過少表現のラベルを持つノードに対し、関数は次を返す:

$$\text{div}(n) = \begin{cases} +0.05 & \text{もし } \text{count}(\text{label}(n)) \leq \frac{1}{2} \max_{\ell} \text{count}(\ell), \\ 0 & \text{それ以外} \end{cases} \quad (1)$$

0.05 という定数は意図的に小さい:scientific score が依然としてランキングを支配し、ボーナスはほぼ同点のものを破るのみである。

LLM フォールバック

LLM が解析不能なインデックスを返した場合、`select_next_node` は決定的なランキングへフォールバックする (`bfts.py:237`、`_fallback_score`):

$$_fallback_score(n) = _scientific_score(n) + \text{div}(n). \quad (2)$$

`has_real_data=True` のノードがあればそれが優先され、フォールバックスコア最大の候補が返される。これにより、LLM 呼び出しが `degrade` してもループは必ず前進する。

`bfts.py:expand`(line 315) は 1 回の呼び出しでちょうど 1 つの新しい子を生成する。同じ親に対して複数の子が欲しい呼び出し側は `expand` を反復的に呼び、これまでに生成された子を `existing_children` 引数経由で渡すことで、LLM が重複方向を提案するのを避ける。

各新子のラベルはハードコードされたテンプレートではなく LLM の判断で決まる; プロンプト (`ari/prompts/orchestrator/bfts_expand.md`) には以下が渡される:

- 親の状態 (`succeeded / failed/no-real-data`) と `_scientific_score`;
- 親の構造化された `node_report` (`delta_vs_parent / concerns / hints`、`node_report/builder.py`) が存在する場合、`planner` が特定の弱点を狙えるよう含める;
- 同じ深さの兄弟スコア、ならびにルートから親までの先祖スコア;
- ツリー全体の多様性指標 (これまでに見たユニークラベル、深さ分布); ならびに
- この親で既に生成された子のラベル (重複提案を避けるため)。

子は `open` で生成され、エージェントループに実行のため渡され (section 3.6)、すべての軸が埋まれば `done` に、エージェントループが例外を投げれば `failed` に遷移する。

3.4 枝刈りと停止

`should_prune`(line 245) は ハード カット オフ のみ を 適用 する: `self.total_nodes >= self.config.max_total_nodes` の時点で以降の拡張

ARI アダプタ

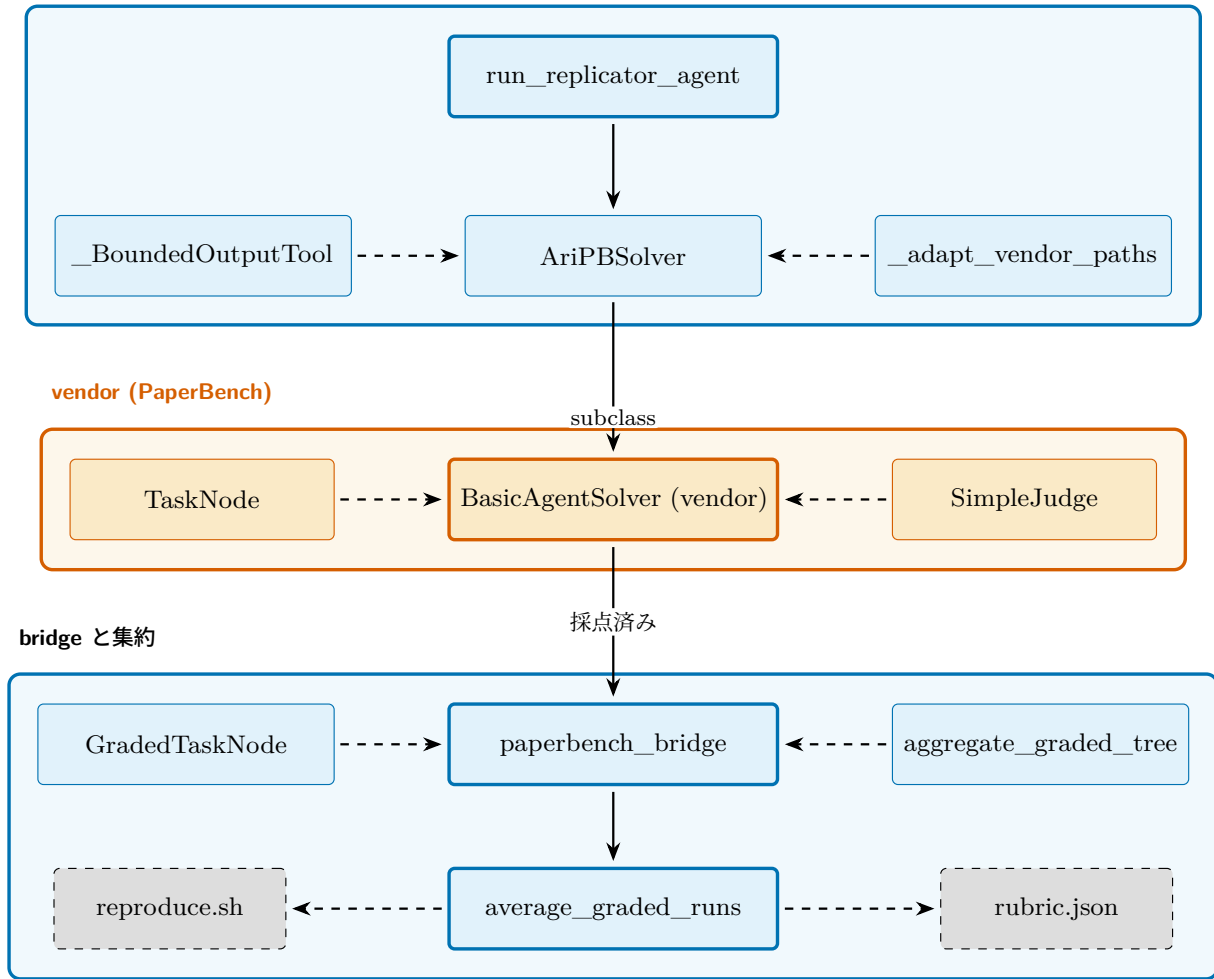


Figure 5: paper-re スキルの構成要素。最上段が ARI のアダプタ層 (ドライバ + ソルバサブクラス + 境界付きツールラッパ + パスアダプタ); 中段が vendoring された PaperBench パッケージ (BasicAgentSolver、TaskNode、SimpleJudge); 下段が submission ごとの GradedTaskNode 木を単一の rubric.json スコアと ARI 側の reproduce.sh に畳み込む bridge である。詳細は section 4.4。

張が止まる。depth は意図的にソフトなシグナルとして扱われ、ハード上限ではなく LLM プロンプトに流されるため、木の深部に現れる高品質ノードが機械的に捨てられない。

ループは次のいずれかが満たされれば停止する:

- グローバル max_total_nodes 上限到達;
- 呼び出し側の cost_tracker.py が強制するステップ/ コスト予算の枯渇;
- 停滞検出器 (lineage_decision.py:334、detect_stagnation) が、_scientific_score の移動最大値がスライディング窓内で改善しないことを観測;
- LineageDecision (lineage_decision.py:149) が当該分岐を明示的に停止。

これらのうちどれが実際に最も頻出するかは経験的問いであり、凍結チェックポイントが揃い次第 section 5 で答える; 本レポートはその分布を主張しない。

3.5 系譜と再現性

系譜とは最良ノードの先祖の連鎖であり、tree.json の lineage キーとしてチェックポイントへ書かれる。LineageState データクラス (lineage_decision.py:64) は BFTS が継続判定に用いる移動統計を保持し、跨チェックポイントの歩行器 lineage.py:126(walk_ancestor_ckpts) は実行を跨ぐ親ポイントを提供する。アイディアプールは lineage.py:157(get_idea_pool_for_ckpt) 経由で継承され、ランキング付きルートアイディアの選択は root_idea_selector.py:39(RootChoice) に記録されて事後監査可能となる。

再現性は三つの不変量にかかる。第一に、すべての乱択操作はノード識別子から自身を播種する。第二に、すべてのツール呼び出しは引数と出力を該当ノードの作業ツリーへ書き、決して親へは書かない —これは MCP 層の cow_node_id チェックで強制される。第三に、コスト台帳 (cost_tracker.py:181、init) がノードあたりのドル額を付与するため、同一プロンプトでの予算検査は決定的となる。

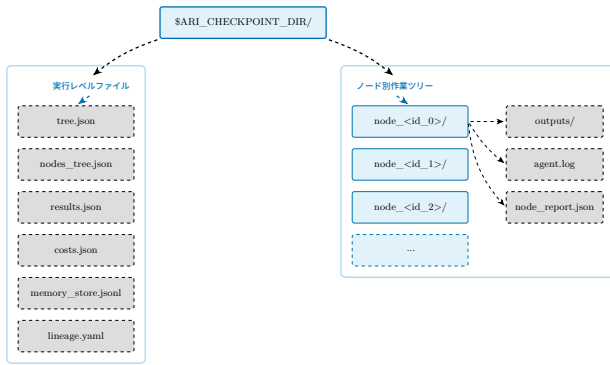


Figure 6: チェックポイントディレクトリのレイアウト。実行レベルファイル (左帯) はオーケストレータが書く; ノード別作業ツリー (右帯) は 1 BFTS 拡張内で生成された成果物を保持する。実行の再現はまさに「このディレクトリに対して再実行する」ことであり、レイアウトはそれ以外の状態が不要であることを保証する。

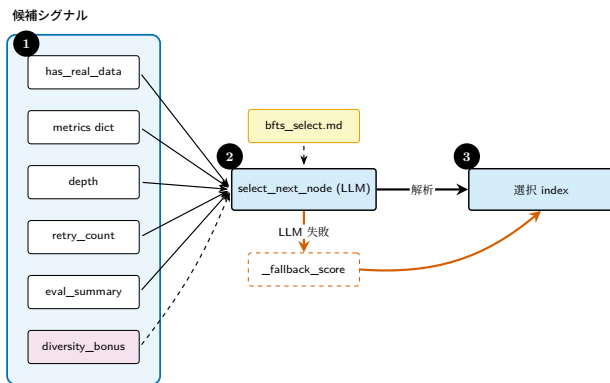
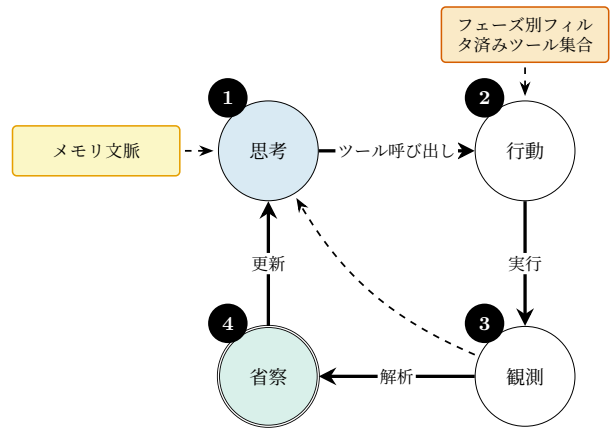


Figure 7: 次に操作するノードを選ぶ LLM に渡される選択シグナル。入力とは構造化された候補記述に連結される; ソフトな `diversity_bonus` はタイプレカであり、主シグナルではない。

3.6 ReAct ドライバ

各ノードの拡張は ReAct 形式のエージェントループ (ari-core/ari/agent/loop.py:77, class AgentLoop) を走らせる。最大ステップ数は `MAX_REACT_STEPS = 80` (loop.py:25) であり、インスタンスごとに `max_react_steps` キーワードで上書きできる。別ガード `MIN_TOOL_CALLS = 2` (line 26) が trivially 短い軌跡を防ぐ。

エージェントが利用できるツール集合は `ari.agent.tool_manager` がフェーズ別にフィルタする; 例えば探索フェーズは論文執筆ツールをマスクし、BFTS 拡張が草稿にショートカットされるのを防ぐ。一部の MCP ツールは ari-core 自身用に予約されており (memory CoW 操作)、LLM からは隠されるため、モデルが任意のノード ID を設定してメモリスキルのコピーオンライトチェックを迂回することはできない (loop.py の docstring 参照)。



`MAX_REACT_STEPS = 80, MIN_TOOL_CALLS = 2`

Figure 8: ReAct ループ。実線が主サイクル; 破線のバックエッジは観測がすでに開いた問いを閉じた場合の早期離脱を表す。

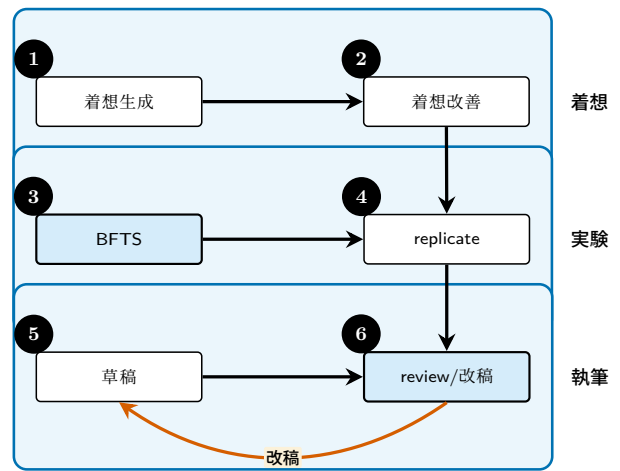


Figure 9: 論文生成スイムレーン: 着想、実験、執筆。各レーンが一つのフェーズに対応する; review/refine から draft へのバックエッジが唯一の非単調遷移である (section 4.3)。

4 論文生成パイプライン

4.1 パイプライン概観

論文生成パイプラインは ARI における二本目の活動の木である (fig. 9)。探索が停止すると、生き残った系譜が `paper` スキルに手渡されて草稿が生成され、続いて `review` スキルが草稿を採点する。実装は ari-core/ari/pipeline/orchestrator.py:131 の `run_pipeline` 関数であり、同ファイル 68 行目の `build_scientific_data` ヘルパが、`paper` スキルが必要とする成果物群をノード単位で集約する。

一回の実行は永続化される 3 つの成果物を生む: `draft.tex` (LaTeX 論文本体)、`review.json` (葉ごとのルブリックスコアと根拠)、そして `rubric.json` (本実行で使用したルブリックインスタンス; ルブリックを動的生成する場合に有用)。

4.2 ルブリック形式

ルブリック R を、葉が組 $(c_i, w_i, judge_i)$ を保持する有限木としてモデル化する。各葉 i は、カテゴリ記述子

c_i 、非負の重み w_i ($\sum_i w_i = 1$ を満たす)、そして段階的スコアを返す判官関数 $\text{judge}_i : P \rightarrow [0, 1]$ を持つ。論文スコアは凸結合

$$\text{score}(P) = \sum_i w_i \text{judge}_i(P). \quad (3)$$

で与えられる。この形式は PaperBench のルブリック木と共有されており、ARI はこれを vendoring された PaperBench パッケージを通じて直接再利用する (section 4.4): 葉型 TaskNode は `paperbench.rubric.tasks` 由来、graded leaf 型 GradedTaskNode は `paperbench.judge.graded_task_node` 由来、重み付き集約エンベロープは `ari-skill-paper-re/src/_paperbench_bridge.py:75` の `aggregate_graded_tree` 由来である。

本番では二種類の判官族を用いる: LLM ベース判官は ARI 用に PaperBench の SimpleJudge (`ari-skill-paper-re/src/_paperbench_bridge.py:54`) をラップして実装する; 決定的判官は二値または数値検証器を持つ項目 (例えば「再現可能性」「ドル絶対値で報告したコスト」) に使う。両者は Evaluator プロトコル (`ari-core/ari/protocols/evaluator.py:19`) を実装する。

4.3 review/refine ループ

改稿サイクルはフィードバック $J(P_t)$ の下で P_t を P_{t+1} へと反復的に書き換える:

$$P_{t+1} = \text{Refine}(P_t, J(P_t)). \quad (4)$$

収束基準 $\text{score}(P_{t+1}) - \text{score}(P_t) < \epsilon$ が連続二回成立した時、またはパイプラインあたり 3 回の上限に達した時にサイクルは停止する。各分岐 — 収束 vs. 上限 — がどの程度の頻度で発火するかは、凍結チェックポイントが揃い次第 section 5 で報告する予定であり、本章では定量主張を行わない。

改稿は意図的に軸ごとに非単調である: 明確性の修正が数値軸を僅かに劣化させ得る。これが fig. 9 のバックエッジの源であり、収束基準は集約スコアにかかわるのであって軸ごとではない。

4.4 paper-re による PaperBench 統合

paper-re スキル (`ari-skill-paper-re`) は ARI の PaperBench 互換複製ドライバである。統合は**再実装ではない**: PaperBench は `vendor/paperbench` に vendoring されて出荷され、これにより ARI は upstream のタスク・ルブリック意味論を逐語的に追跡する (配置要件は PaperBench の `rubric.md` §8 に文書化されている)。3 つの統合点を述べる。

1. ソルバ サブクラス: AriPBSolver

`ari-skill-paper-re/src/_replicator_agent.py:269` は PaperBench upstream の `paperbench.solvers.basic.BasicAgentSolver` をサブクラス化する。chz の immutable-config 規則により新フィールドが追加されなくてもサブクラスにデコレータが必要となるが、ARI のオーバーライドは最小限である:

- `_get_tools():submit` 以外の全ツールを `_BoundedOutputTool` でラップし、`bash / python / file-read / search` の出力を次の API 呼び出しへ流す前にサイズ制限する。`submit` は名前で dispatch され、`execute()` が呼ばれることがないので、ラップされずにそのまま通過する。
- `_run_agent():`
`paperbench.solvers.utils.sanity_check_docker` を `_bypass_docker_sanity_check` で迂回し、`upstream` のハードコード `/home/{submission,paper}` パスを ARI のワークスペース相対レイアウト (`LocalComputer` `cwd = repro_sandbox/`、論文を `paper/` に、`submission` を `submission/` に置く) へ書き換える。プロンプト内容はその他の点で `upstream` の `get_instructions / get_system_message` から逐語的に取得され、ARI は `upstream` の複製タスク枠組み (7 日予算、「すべての結果を複製するまで停止しない」等) に整合する。

`AriPBSolver` はまた `upstream` の `check_for_existing_run` を保持しており、部分的に完了した実行を幕等に再開できる。

2. ルブリック駆動の成果物リスト

vendor PaperBench は ARI の論文ごとのルブリック木を知らないため、`AriPBSolver._run_agent` は `LocalPBTask.rubric_expected_artifacts` が空でない場合、ユーザ指示に `EXPECTED_ARTIFACTS` ブロックを追記する。追記されるブロックは、成功した `reproduce.sh` が産出すべきパスを列挙する; これがないと、grader の葉主張はどのファイルを検証すべきかのヒントを欠く。ルブリック木自体は `ari-skill-paper-re/src/_paperbench_bridge.py:63` (`task_node_from_dict`) で構築される。

3. ドライバ: run_replicator_agent

`ari-skill-paper-re/src/_replicator_agent.py:365` が非同期エントリポイントである。標準の `build_reproduce_sh` エンベロープ `{populated, output_dir, files, expected_artifacts, max_runtime_sec, model, prompt_sha256, notes, warnings}` を返す。`output_dir` はエージェントのワークスペースであると同時に、ARI が rollout 後に `reproduce.sh` を見つけることを期待する場所でもある; `upstream` のソルバは `agent.log` 等の bookkeeping を `run_dir` に書き込むため、これをワークスペース自身に向けることで成果物が `reproduce.sh` と隣接する。

既定の `completer` は `OpenAIResponsesTurnCompleterConfig` でモデルは `gpt-5-mini` (環境変数 `ARI_MODEL_REPLICATOR` で上書き可能); `LiteLLM` ルートの代替は `_litellm_completer.py` のヘルパ経由で配線される。既定の時間上限は実行あたり 12 時間; `sandbox_kind` は `auto` で、ローカル Apptainer イメージが供給されていれば fallback する。

4. 採点と集約

複製エージェントが完了すると、採点は `ari-skill-paper-re/src/_paperbench_bridge.py` の `judge_submission` に委譲される。この関数は ARI の (`paper_md`, `submission_dir`, `reproduce_log`, `judge_model`) タプルを `upstream` の `SimpleJudge` に適合させ、`submission` ごとに `GradedTaskNode` 木を生成する。実行間の集約は次の二つのヘルパで処理される:

- `aggregate_graded_tree(line 75)`: 一つの `GradedTaskNode` をルブリック重み w_i 経由で重み付きスカラーへ畳み込み、カテゴリ別内訳を併せて返す。
- `average_graded_runs(line 91 周辺)`: n 個の `GradedTaskNode` 木に対する実行ごとの平均を取り、ノイズ低減のためエージェントを再走させた場合でも単一の複製スコアが報告される。

二段階設計 — 葉採点には `vendor` の `SimpleJudge`、形状と集約には ARI のアダプタ — により、`PaperBench upstream` のアップグレードは `vendor swap` となり (ARI コード変更不要)、ARI は葉採点を単一数値へ結合する仕方を完全にコントロールする。同じエンベロップが葉が二値のとき $judge_i \in \{0, 1\}$ として `eq. (3)` に流れる。

4.5 故障モードとガードレール

頻出する故障モードを 3 つ命名する:

- **LLM のみの裏付け**: 論文が、系譜中のいずれのノードも裏付けない主張をする。緩和策は静的検査: 数値主張または因果主張は、ノード成果物への `\code{...}` 参照経由で必ず辿れねばならない。
- **引用ハルシネーション**: 論文が存在しない論文を引用する。緩和策は構造的: LLM が引用キーを発明することを禁じ、すべての `bib` エントリは `Semantic Scholar` に対して検証されなければならない (`section 6`)。
- **改稿ドリフト**: スコアは上昇するが論文が一貫性を失う。緩和策はルブリックの軸ごと下限: いずれかの軸が 0.5 を下回れば改稿は中止される。

草稿時のソースファイル選択は `node_selection.py:204 (select_source_files_for_publication)` が担う; 主張が頼ることのできる系譜ノードを正確に列挙することで、LLM のみの裏付け型の故障を防ぐ。コンパニオンの `load_selected_sources(line 258)` が (`node_id`, `rel_path`) 各組のバイトを読み、オプションの `size_budget` を尊重するので、論文草稿が `context` を超えることは決してない。

4.6 複製器のためのツール出力境界

複製エージェントは単一の `bash` セッション内で何時間も過ぎ得る; ツール出力 (長い `stdout`、大規模ディレクトリリスト) は放置すれば後続の各プロンプトを支配する。ARI は `_BoundedOutputTool (ari-skill-`

`paper-re/src/_replicator_agent.py:243)` でこれを境界し、同ツールが内部ツールの `execute()` をラップして出力を `_truncate_tool_output(line 218)` で構成可能なバイト上限へ後処理する。これが `PaperBench upstream` の自由形式トランスクリプトと ARI の境界付き再生可能なものとの違いであり、複数時間の rollout でもプロンプト予算を予測可能に保つ理由である。

5 評価 (保留)

本章は意図的に短い。本スナップショットでは評価を実施していない。本レポートの旧草稿は具体的な中央値や葉ごとのスコアを記載していたが、見直したところ、それらの数値は R3a 執筆時に書かれた説明用例示であり、凍結された ARI チェックポイントからの計測ではないことが判明した。そのため、それらは削除した。実在する成果物のみを残す。

実装済みのもの:

- コスト計上機構: `CostTracker (cost_tracker.py:77)` が各モデル呼び出しに対してドルコストを付与する。 `init` は 181 行目でトラックをチェックポイントディレクトリに繋ぐ; 結果として `cost_usd` フィールドが `results.json` に実行末に書かれる。
- 探索ループは停滞検出器 (`section 3.4`) により終了する; $T_{\max} / C_{\max}$ ではなく停滞が支配する経験的頻度を本章で報告する予定 — スナップショットが揃った時点で。
- PGF 図の生成スクリプト (`shared/figures/scripts/`) はシードが固定されている。 `makefigures` は決定的な合成プレースホルダを再生成し `check_figures` を通過させるが、それらのプレースホルダは本文での証拠としては引用していない。

未実装のもの:

- ARI v0.7.1 の凍結チェックポイントの確定と、 `tree.json / results.json` の `shared/figures/data/` への抽出。現状、この `data` ディレクトリには `*.meta.yaml` のみが置かれ、いずれも `source.synthetic:true` とマークされている。
- 中央値の実行コスト、シード別の探索曲線、実レビューに基づくカテゴリ別ルブリック分布の報告。これらは凍結チェックポイントを上陸させ集約値を再導出する後続 PR で本章に現れる予定である。
- 実対象論文に対する `paper-re` スキルからの `PaperBench` 風の再現スコアの報告。

これらの後続作業が上陸するまで、本章は定量的主張を一切含まない。他章からの相互参照 (例えば `section 3.4` 内) は、経験的証拠が将来配置される場所を指し示すのみであり、現在置かれている場所を指してはいない。

6 関連研究

ARI を 4 つの先行研究の流れに対して位置付ける。

6.1 コードのための木探索

AIDE [2] は ML 工学を計画・コード・デバッグ状態上の木探索として形式化する; AlphaCode [3] は競技プログラミングに対する大規模生成・濾過の規範的参照であり; MLE-bench 評価器 [1] はいまや ML 工学エージェントを計測する標準である。ARI は AIDE から BFTS の骨格を継承するが、(i) LLM 駆動の候補選択器 (section 3.2) — 閉形式のスカラ効用を固定せず、選択プロンプトが `has_real_data`、`metrics` 全体、深さ、リトライ回数、ソフト `diversity_bonus` を一括重み付けできる —、および (ii) 実行間の系譜 (section 3.5) — AIDE も MLE-bench も形式化していない — を追加している。

6.2 自律研究エージェント

AI Scientist ファミリー [4] は同様のループ (idea → experiment → paper) をエンドツーエンドで閉じるが、ルブリックと改稿サイクルを第一級オブジェクトとして露出させない。VirSci [9] は複数の科学者ペルソナを模擬し、彼らの合意で出力を採点する; Agent Laboratory [6] はエージェントを駆動者ではなく協働者として扱う。ARI は二点の運用面で異なる: **すべてがチェックポイントの中で生きる** (P1 / P4)、そしてルブリックが探索と執筆の間の契約である。

6.3 論文評価

PaperBench [8] は我々の `paper-re` 統合に最も近い類例である; それはルブリック・アズ・ツリー形式の先駆者であった。我々は同じ葉ごとの採点と集約重み (eq. (3)) を採用するが、改稿ガードとして使う軸ごとの下限で拡張している (section 4.5)。

6.4 基礎

エージェントループは ReAct パターン [11] に従う; 改稿サイクルは Self-Refine [5] と Reflexion [7] の系譜にある; **思考ステップ**で LLM に逐次的なスクラッチパッドを与えるという選択は Chain-of-Thought [10] の系譜にある。これらの参照のいずれもチェックポイントスコープの状態を提案していない; それが ARI の貢献である。

初期の草稿は別の ML 研究エージェントベンチマークも引用していたが、いずれの候補ソースも我々の検証規則 (section 4.5) を通らないことが明らかになった時点で、その参照を取り除いた; 関連研究の網羅性は S2 で検証可能なエントリのみを保つ。

7 議論と限界

7.1 決定性 vs 新規性

P2、決定性原則は本システムの拘束的制約である。P2 を厳密に強制したことが、`ari-skill-memory` に「LLM 呼び出しなし」を宣言させている — それは検索の採点を、出力が遠隔サービスに依存する埋め込みモデルではなく、決定的なキーワード関数で行わなければならない。代償は、検索の質がキーワード採点で復元できる範囲に上界を持つことである。

これは偶然ではない: 再現性の勝利のすべてに、非決定性に対して扉を閉じるという代償が伴う。下流ユーザとの契約は実行ではなくチェックポイントであるため、我々はこの代償を受け入れる。

7.2 スケーリング限界

BFTS スケジューラの基盤にある単一マシン仮定は最大のスケーリング制約である。 $b > 10$ の並列拡張を望む実行には別のスケジューラが必要である; `ari-core/ari/orchestrator/scheduler.py` の現行スケジューラは同じ親の拡張を直列化する。マルチホスト拡張には系譜規律 (P4) を、先祖歩行のみではなくマージ操作も扱えるよう拡張する必要がある。

別の制約は LLM 採点者のコストである。経験的には採点者は探索者と並ぶ支配的な費用項目の一つだが、絶対ドル額で第 2 位となるか否かは凍結チェックポイントが揃ってから section 5 が答えるべき問いである。(論文ハッシュ、ルブリックハッシュ) で採点者出力をキャッシュすることは助けになる; 我々はまだこれを出荷していない。

7.3 未解決問題

1. **Off-policy 改稿**。現行の改稿サイクルは執筆と同じモデルを使う。執筆と改稿を分離して別々のモデルを使えるようにすれば、より安価なモデルが粗い構造的書き換えを担当できる。
2. **チェックポイント間のルブリック継承**。子チェックポイントは既定で親のルブリックを再利用できるべきだが、現状ではルブリックは再生成される。
3. **ツール失敗の復旧**。現状ではツール呼び出しの失敗はノードを `failed` にマークする。劣化したツール (例えば小さなモデル) で再試行する復旧方策があれば、長裾コストが減る。

これら 3 件はいずれも対応可能であり、各々 v0.8 までに 1 PR で着地できる見込みである。

A 記号表

下表は `shared/notation.tex` で定義されたマクロを反映する。本文で使われる各記号はここに必ず現れる; 静的検査 (`check_notation.py`) が、本文中に未定義の記号がないかを検証する。記号は数式モードと地の文の双方で使えるよう `\ensuremath{}` でラップされている; 新たな記号を導入する際は、まずこのファイルへ追加し、対応するマクロを `notation.tex` に登録してから本文で参照すること。

記号	意味
$\mathcal{N}$	探索木のノード集合
$\mathcal{T}$	探索木 $(\mathcal{N}, E)$
$n \in \mathcal{N}$	個別のノード
$\text{ch}(n)$	n の子集合
$\text{pa}(n)$	n の親
$\text{depth}(n)$	n の深さ
$s(n)$	ノード状態 $\in \{\text{open, eval, done, failed}\}$
$r(n)$	n のスカラー報酬
$\mathbf{e}(n)$	軸ごとの評価ベクトル
ϕ	軸 $\rightarrow$ スカラー集約器
$\text{div}(n)$	多様性ボーナス (eq. (1))
$\text{_fallback_score}(n)$	決定的フォールバックランキング (eq. (2))
$T_{\max}, C_{\max}$	ステップ・コスト予算
R	ルブリックの木
c_i	葉 i のカテゴリ
w_i	葉 i の重み
judge_i	葉 i の判官
P	論文草稿
$\text{score}(P)$	論文集約スコア (eq. (3))
Refine	改稿演算子 (eq. (4))
ckpt	チェックポイントディレクトリ
$\text{lin}(n)$	n で終端する系譜の連鎖

- [7] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: 2303.11366 [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [8] Giulio Starace et al. *PaperBench: Evaluating AI’s Ability to Replicate AI Research*. 2025. DOI: 10.48550/arXiv.2504.01848. arXiv: 2504.01848 [cs.AI]. URL: <https://arxiv.org/abs/2504.01848>.
- [9] Haoyang Su et al. *Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System*. 2024. DOI: 10.18653/v1/2025.acl-long.1368. arXiv: 2410.09403 [cs.CL]. URL: <https://arxiv.org/abs/2410.09403>.
- [10] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [11] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: <https://arxiv.org/abs/2210.03629>.

References

- [1] Jun Shern Chan et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. 2024. DOI: 10.48550/arXiv.2410.07095. arXiv: 2410.07095 [cs.LG]. URL: <https://arxiv.org/abs/2410.07095>.
- [2] Zhengyao Jiang et al. *AIDE: AI-Driven Exploration in the Space of Code*. 2025. arXiv: 2502.13138 [cs.AI]. URL: <https://arxiv.org/abs/2502.13138>.
- [3] Yujia Li et al. *Competition-Level Code Generation with AlphaCode*. 2022. DOI: 10.1126/science.abq1158. arXiv: 2203.07814 [cs.PL]. URL: <https://arxiv.org/abs/2203.07814>.
- [4] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. *The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery*. 2024. arXiv: 2408.06292 [cs.AI]. URL: <https://arxiv.org/abs/2408.06292>.
- [5] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. arXiv: 2303.17651 [cs.CL]. URL: <https://arxiv.org/abs/2303.17651>.
- [6] Samuel Schmidgall et al. *Agent Laboratory: Using LLM Agents as Research Assistants*. 2025. DOI: 10.18653/v1/2025.findings-emnlp.320. arXiv: 2501.04227 [cs.AI]. URL: <https://arxiv.org/abs/2501.04227>.